Généralités

Tout replier

Annonces

Cours

cours python base

cours socket

cours scapy

fiche résumé scapy

fiche résumé NDIS

bibliothèques chiffrement

liste honeypot

résumé commandes scapy

Sujet du projet

sujet

Documentation

chapitre de mon livre sur les bases python (python 2)

chapitre de mon livre sur python en reseau (python 2)

Comptes rendus journaliers

?

Séance 1 : Introduction, honeypot et Script Python (3h)

Introduction, répartition des tâches et planning (1h)

Objectifs :

- Comprendre les objectifs du projet.
- Définir les rôles et responsabilités de chaque membre de l'équipe.
- Élaborer un planning pour structurer le travail à réaliser.

Étapes détaillées :

1. **Relecture du sujet et des objectifs du projet :**
    - **Contexte du projet :** Prendre connaissance du contexte et des enjeux du projet, notamment l'utilisation du honeypot et le développement d'un script Python pour le pentest automatisé.
    - **Objectifs spécifiques :** Clarifier les objectifs précis à atteindre, tels que la récupération et l'analyse des données du Honeypot.
2. **Définition des rôles de chaque membre du groupe :**
    - **Exemple de répartition des rôles :**
        - **Chef de projet :** Responsable de la coordination générale et du respect des délais.
        - **Développeur Python :** Chargé de la rédaction et des tests du script Python.
        - **Analyste de données :** Spécialiste de l'analyse et de l'interprétation des données récupérées.
    - **Discussion :** S'assurer que chaque membre comprend bien son rôle et ses responsabilités.
3. **Établissement d'un planning prévisionnel et réel :**
    - **Planning prévisionnel :** Définir les principales étapes du projet avec des dates butoirs pour chaque tâche.
        - **Exemple :**
            - Semaine 1 : Installation et prise en main du honeypot.
            - Semaine 2 : Développement du script Python.
            - Semaine 3 : Exécution du script et analyse des données.
    - **Planning réel :** Ajuster le planning en fonction des progrès réels et des éventuels imprévus.
    - **Outils :** Utiliser des outils de gestion de projet tels que Trello ou Asana pour suivre l'avancement des tâches.

Installation et prise en main du honeypot (1h)

Objectifs :

- Installer le honeypot dans un environnement Docker.
- Se familiariser avec l'interface et les fonctionnalités du honeypot.

Étapes détaillées :

1. **Installation du honeypot dans un environnement Docker :**
    - **Pré-requis :** S'assurer que Docker est installé et configuré sur la machine hôte.
    - **Téléchargement de l'image du honeypot :**
    - **Lancement du honeypot :**

      bash
      ```
      docker run -d --name nom_honeypot -p 6400:6400 -p 6429:6429 -p 6430:6430 -p 8080:8080 dtagdevsec/tpotce
      ```

    - **Vérification de l'installation :**
        - Utiliser la commande `docker ps` pour vérifier que le conteneur est en cours d'exécution.

          bash
          ```
          docker ps
          ```

2. **Familiarisation avec l'interface et les fonctionnalités du honeypott :**
    - **Accès à l'interface web :** Ouvrir un navigateur et accéder à l'interface T-pot via l'URL fournie (par exemple, `http://localhost:8080`).
    - **Exploration des fonctionnalités :**
        - **Tableau de bord :** Comprendre les principales métriques et visualisations disponibles.
        - **Logs et événements :** Apprendre à naviguer et interpréter les logs d'événements générés par le Honeypot.
        - **Configuration :** Voir comment configurer et personnaliser les fonctionnalités de T-pot.

Début du développement du script Python de pentest automatisé (1h)

Objectifs :

- Choisir les outils de pentest à utiliser et les attaques à simuler.

- Commencer le développement du script Python.

Étapes détaillées :

1. **Choix des outils de pentest à utiliser et des attaques à simuler :**
    - **Outils de pentest courants :**
        - **Nmap :** Scanner réseau pour découvrir les hôtes et services sur un réseau.

          bash
          ```
          import os
          os.system('nmap -sP 192.168.1.0/24')
          ```
        - **Metasploit :** Plateforme pour développer et exécuter des exploits contre des cibles distantes.

          bash
          ```
          from metasploit.msfrpc import MsfRpcClient
          client = MsfRpcClient('password')
          ```
    - **Attaques à simuler :**
        - **Scan de ports :** Utiliser Nmap pour scanner les ports ouverts sur la cible.
        - **Exploits :** Utiliser Metasploit pour tenter d'exploiter des vulnérabilités connues.
2. **Début du codage du script en Python :**
    - **Importation des bibliothèques nécessaires :**

      python
      ```
      import subprocess
      ```
    - **Exemple de code pour lancer un scan de ports avec Nmap :**

      python
      ```
      def scan_ports(ip_range):
          scan_result = subprocess.run(['nmap', '-sP', ip_range], capture_output=True, text=True)
          return scan_result.stdout

      # Exemple d'appel de fonction
      print(scan_ports('192.168.1.0/24'))
      ```
    - **Gestion des erreurs potentielles :**
        - Utiliser des blocs `try-except` pour gérer les exceptions et les erreurs d'exécution.

          python
          ```
          try:
              result = scan_ports('192.168.1.0/24')
              print(result)
          except Exception as e:
              print(f"Erreur lors du scan des ports : {e}")
          ```

Etat avancement du 27 Novembre

Ouvert le : jeudi 27 novembre 2025, 10:00 **À rendre :** jeudi 27 novembre 2025, 17:00

Vous rendez ici un pdf , en donnant votre état d'avancement de la session.

- nom des membres du groupe

- répartition des taches

- planning prévisionnel

- outils utilisés (IDE, bibliothèques python, moyens de communication dans le groupe)

-Etat d'avancement du code, test réalisés (si déjà faits)

-travail prévu pour la session du 15 Décembre

?

Séance 2 : Script Python, Tests et Analyse (3h)

Finalisation du script Python (1h)

Objectifs :

- Intégrer la fonctionnalité de récupération des données du Honeypot dans le script.
- Réaliser des tests et déboguer le script pour s'assurer de son bon fonctionnement.

Étapes détaillées :

1. **Intégration de la récupération des données du Honeypot dans le script :**

   - **Récupération des données :** Utiliser des bibliothèques comme `requests` ou `http.client` pour faire des requêtes HTTP au Honeypot et récupérer les données collectées.
   - **Exemple de code :**

     python
     ```
     import requests
     # URL de l'API du Honeypot
     honeypot_url = 'http://honeypot.local/api/data'
     # Requête GET pour récupérer les données
     response = requests.get(honeypot_url)
     if response.status_code == 200:
         honeypot_data = response.json()
         print("Données récupérées :", honeypot_data)
     else:
         print("Échec de la récupération des données")
     ```

2. **Tests et débogage du script :**

   - **Tests unitaires :** Écrire des tests unitaires pour vérifier chaque partie du script.
   - **Exemple de test unitaire :**

     python
     ```
     import unittest
     from script import fetch_honeypot_data

     class TestHoneypotData(unittest.TestCase):
         def test_fetch_honeypot_data(self):
             data = fetch_honeypot_data()
             self.assertIsNotNone(data)
             self.assertIn('attacks', data)

     if __name__ == '__main__':
         unittest.main()
     ```

   - **Débogage :** Utiliser des outils de débogage comme `pdb` (Python Debugger) pour identifier et corriger les erreurs dans le script.

     python
     ```
     import pdb; pdb.set_trace()  # Déclenchement du mode débogage
     ```

Exécution du script sur le honeypot et récupération des données (1h)

Objectifs :

- Lancer le script sur la plateforme du honeypot.
- Vérifier que les données sont correctement récupérées et enregistrées.

Étapes détaillées :

1. **Lancement du script sur le Honeypot :**

   - **Connexion au honeypot :** Utiliser SSH pour se connecter au serveur où le Honeypot est déployé.

     bash
     ```
     ssh user@tpot_server
     ```

   - **Exécution du script :** Lancer le script Python directement sur le serveur.

     bash
     ```
     python3 honeypot_script.py
     ```

2. **Vérification de la bonne récupération et de l'enregistrement des données :**

   - **Logs de l'exécution :** Examiner les logs produits par le script pour s'assurer que les données sont bien récupérées.

?

- Exemple de log :

```
[INFO] Récupération des données du Honeypot...
[INFO] Données récupérées : {'attacks': [...], 'details': [...] }
[INFO] Données enregistrées avec succès.
```

- **Vérification des fichiers :** Confirmer que les données récupérées sont enregistrées correctement dans un fichier ou une base de données.
  - Exemple de vérification :

    bash
    ```
    cat honeypot_data.json  # Affiche le contenu du fichier JSON contenant les données
    ```

Analyse des données du Honeypot (1h)

Objectifs :

- Analyser les données collectées après l'exécution du script.
- Identifier les tentatives d'attaques et les techniques utilisées.

Étapes détaillées :

1. **Analyse des données collectées après l'exécution du script :**
   - **Chargement des données :** Utiliser des bibliothèques comme `pandas` pour charger et manipuler les données collectées.

     python
     ```
     import pandas as pd
     data = pd.read_json('honeypot_data.json')
     print(data.head())
     ```

2. **Identification des tentatives d'attaques et des techniques utilisées :**
   - **Filtrage et regroupement :** Utiliser les fonctionnalités de `pandas` pour filtrer et regrouper les données par type d'attaque, adresse IP source, etc.

     python
     ```
     attacks = data['attacks']
     attack_types = attacks['type'].value_counts()
     print("Types d'attaques les plus fréquents :", attack_types)
     ```

   - **Exemples de techniques :**
     - Analyse de la fréquence des tentatives de connexion par adresse IP.

       python
       ```
       ip_attempts = attacks['source_ip'].value_counts()
       print("Adresses IP les plus actives :", ip_attempts)
       ```

     - Détection des méthodes d'attaque les plus courantes (ex : bruteforce, exploitation de vulnérabilités spécifiques).

       python
       ```
       methods = attacks['method'].value_counts()
       print("Méthodes d'attaque les plus utilisées :", methods)
       ```

?

Problème Courant avec Outlook/Office 365

Cette erreur est courante avec Outlook/Office 365 car Microsoft a désactivé l'authentification "basique" pour des raisons de sécurité. Pour continuer à envoyer des emails, vous pouvez utiliser l'une des deux solutions suivantes :

Solution Recommandée : Utiliser un "Mot de Passe d'Application"

1. **Connectez-vous à votre compte Microsoft** :
   - Allez sur le site Microsoft et connectez-vous à votre compte.
2. **Allez dans Paramètres** :
   - Cliquez sur votre profil ou votre avatar en haut à droite.
   - Sélectionnez **Sécurité** ou **Paramètres de Sécurité**.
3. **Sécurité Avancée** :
   - Accédez à la section **Sécurité avancée**.
4. **Activez l'authentification à deux facteurs** :
   - Si ce n'est pas déjà fait, activez l'authentification à deux facteurs (2FA).
5. **Créez un "Mot de Passe d'Application"** :
   - Créez un mot de passe d'application spécifique pour votre script ou logiciel qui envoie des emails.

Alternative : Utiliser OAuth2 pour l'Authentification

Vous pouvez également utiliser OAuth2 pour l'authentification. Voici un exemple de script Python pour envoyer un email en utilisant OAuth2 avec la bibliothèque O365 :

python

```python
from O365 import Account
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

def envoyer_email_oauth2(destinataire, sujet, message):
    # Remplacez avec vos identifiants d'application Microsoft
    client_id = "votre_client_id"
    client_secret = "votre_client_secret"

    credentials = (client_id, client_secret)
    account = Account(credentials)

    if account.authenticate():
        mailbox = account.mailbox()
        m = mailbox.new_message()
        m.to.add(destinataire)
        m.subject = sujet
        m.body = message

        try:
            m.send()
            print("Email envoyé avec succès!")
        except Exception as e:
            print(f"Erreur: {e}")
    else:
        print("Échec de l'authentification")

# Exemple d'utilisation
if __name__ == "__main__":
    destinataire = "adresse@example.com"
    sujet = "Test Email"
    message = "Bonjour,\n\nCeci est un test.\n\nCordialement"

    envoyer_email_oauth2(destinataire, sujet, message)
```

Pour utiliser cette solution, voici les étapes à suivre :

1. **Installer la bibliothèque O365** :
   - Exécutez `pip install O365` dans votre terminal.
2. **Créer une Application dans le Portail Azure** :
   - Allez sur Azure Portal.
   - Créez une nouvelle application.
   - Obtenez le `client_id` et le `client_secret`.

?

- Configurez les permissions appropriées (ex. : `SMTP.Send`).

Option Plus Simple : Utiliser Gmail avec un Mot de Passe d'Application

Si vous préférez une solution plus simple, utilisez Gmail qui permet encore l'authentification simple avec un mot de passe d'application. Voici un exemple de script Python pour envoyer un email via Gmail :

python

```python
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

def envoyer_email_gmail(destinataire, sujet, message):
    expediteur = "votre_email@gmail.com"
    mot_de_passe = "votre_mot_de_passe_application"
    serveur_smtp = "smtp.gmail.com"
    port = 587

    email = MIMEMultipart()
    email['From'] = expediteur
    email['To'] = destinataire
    email['Subject'] = sujet
    email.attach(MIMEText(message, 'plain'))

    try:
        with smtplib.SMTP(serveur_smtp, port) as serveur:
            serveur.starttls()
            serveur.login(expediteur, mot_de_passe)
            serveur.send_message(email)
        print("Email envoyé avec succès!")
    except Exception as e:
        print(f"Erreur: {e}")
```

Ce script se connecte à Gmail en utilisant un mot de passe d'application et envoie un email.

Séance 3 : NIDS et Début de la Vidéo (3h)

Objectif :

Cette séance a pour but d'approfondir les concepts de systèmes de détection d'intrusion réseau (NIDS) et d'enseigner l'utilisation avancée de Scapy pour capturer et analyser les paquets réseau. Les étudiants commenceront à développer leur propre NIDS en Python et prépareront le contenu de leur vidéo de soutenance.

Partie 1 : Introduction au NIDS (1h)

Objectif : Comprendre le fonctionnement des NIDS et les types d'attaques réseau courantes.

Comprendre le fonctionnement des NIDS

1. **Présentation des NIDS** :
    - **Définition** : Un Système de Détection d'Intrusion Réseau (NIDS) surveille le trafic réseau pour détecter des activités malveillantes.
    - **Exemples** : SNORT, Suricata.
    - **Fonctionnement** :
        - **Inspection des paquets** : Analyse du contenu des paquets réseau.
        - **Règles de détection** : Utilisation de règles définies pour identifier des motifs d'attaque.
        - **Alertes** : Génération d'alertes lorsqu'une attaque est détectée.
2. **Types d'attaques réseau courantes** :
    - **Scan de ports** : Détection des services ouverts sur une machine.
    - **Attaques par déni de service (DoS)** : Inondation d'un réseau pour le rendre indisponible.
    - **Injection SQL** : Envoi de requêtes SQL malveillantes pour exploiter les bases de données.

Exercice :

- Discutez des scénarios où un NIDS serait crucial.
- Analysez les logs d'attaques réelles et identifiez les signes avant-coureurs d'intrusion.

Partie 2 : Développement du NIDS avec Scapy (1h)

Objectif : Débuter le développement d'un NIDS en Python en utilisant Scapy pour capturer et analyser des paquets réseau en temps réel.

Capture des paquets réseau en temps réel

Exercice :

1. **Capture des paquets** :
    - **Script de capture de paquets** :

      python
      ```
      from scapy.all import sniff

      def capture_paquet(paquet):
          # Simplified packet information for now
          print(f"Paquet capturé : {paquet.summary()}")

      # Capture des paquets en temps réel
      sniff(prn=capture_paquet)
      ```

2. **Stockage des paquets capturés** :
    - **Exemple de stockage dans un fichier** :

      python
      ```
      from scapy.all import sniff, wrpcap

      paquets_captures = []

      def capture_paquet(paquet):
          paquets_captures.append(paquet)

      # Capture 50 paquets et les stocke dans un fichier PCAP
      sniff(prn=capture_paquet, count=50)
      wrpcap("captured_packets.pcap", paquets_captures)
      ```

Analyse des paquets pour identifier des caractéristiques potentiellement malveillantes

Exercice :

1. **Détection de scans de ports** :

- **Script de détection** :

  python
  ```
  from scapy.all import sniff

  scan_ports = {}

  def detecte_scan(paquet):
      if paquet.haslayer("TCP"):
          ip_src = paquet["IP"].src
          dport = paquet["TCP"].dport
          if ip_src not in scan_ports:
              scan_ports[ip_src] = []
          scan_ports[ip_src].append(dport)
          if len(scan_ports[ip_src]) > 10:  # Seuil de détection
              print(f"Scan de ports détecté depuis {ip_src}")

  sniff(prn=detecte_scan)
  ```

2. **Détection d'attaques par déni de service (DoS)** :

   - **Script de détection** :

     python
     ```
     from scapy.all import sniff

     requetes = {}

     def detecte_dos(paquet):
         if paquet haslayer("IP"):
             ip_src = paquet["IP"].src
             if ip_src not in requetes:
                 requetes[ip_src] = 0
             requetes[ip_src] += 1
             if requetes[ip_src] > 1000:  # Seuil de détection
                 print(f"Attaque DoS détectée depuis {ip_src}")

     sniff(prn=detecte_dos)
     ```

Pistes de réflexion et solutions :

- **Pourquoi utiliser des seuils dans la détection ?**
  - Cela permet de réduire les faux positifs en s'assurant que seules les activités réellement suspectes sont signalées.
- **Comment affiner les règles de détection ?**
  - En analysant des logs réels et en ajustant les seuils et les règles en fonction des modèles d'attaque observés.

Partie 3 : Début de la Création du diaporama de Soutenance (1h)

Objectif : Préparer les diapositives et enregistrer les premières parties de la vidéo de soutenance.

Préparation des diapositives

1. **Diapositives pour l'introduction** :
   - Présenter le projet de NIDS.
   - Objectifs et portée du projet.
   - Matériel et logiciels utilisés (Python, Scapy, Docker, Kubernetes).
2. **Présentation de Honeypot** :
   - Description de Honeypot.
   - Fonctionnalités et avantages.
   - Intégration avec le NIDS développé.
3. **Présentation du script Python et des résultats obtenus** :
   - Explication du script de capture et d'analyse des paquets.
   - Démonstration des détections effectuées (scans de ports, attaques DoS).
   - Présentation des résultats et de leur interprétation.

Exercice :

- Rédigez un script narratif pour chaque diapositive pour assurer une présentation claire et concise.

Création des premières parties de la soutenance

1. **Introduction** :

?

- Enregistrer une présentation verbale du projet, des objectifs et des outils utilisés.

2. **Présentation de Honeypot et du script Python** :
    - Enregistrer des démonstrations de l'utilisation de Honeypot.
    - Enregistrer une explication détaillée du script Python.
3. **Démonstration des résultats** :
    - Enregistrer des segments vidéo montrant les résultats de l'analyse des paquets et la détection des attaques.

Pistes de réflexion et solutions :

- **Comment rendre la soutenance engageante ?**
    - Utilisez des illustrations visuelles, des exemples concrets et une narration énergique pour maintenir l'intérêt du spectateur.

Conclusion

Cette séance offre une introduction complète aux concepts de NIDS et à l'utilisation avancée de Scapy pour capturer et analyser des paquets réseau. Les étudiants débuteront le développement de leur propre NIDS en Python et prépareront le contenu de la presentation de la soutenance.

Séance 4 : Finalisation du NIDS, soutenance et Rendu (3h)

Objectif :

Cette séance a pour but de finaliser le développement du système de détection d'intrusion réseau (NIDS), de préparer la vidéo de soutenance et de vérifier que tous les éléments du projet sont complets et fonctionnels.

Partie 1 : Finalisation du NIDS (1h)

Objectif : Implémenter les méthodes de détection d'attaques spécifiques, mettre en place un système d'alertes et intégrer l'envoi de rapports chiffrés par mail. Effectuer des tests et ajuster le NIDS.

Implémentation des méthodes de détection d'attaques spécifiques

Exercice :

1. **Détection des tentatives de brute force** :
    - **Script de détection** :

      python
      ```
      from scapy.all import sniff

      brute_force_attempts = {}

      def detecte_brute_force(paquet):
          if paquet.haslayer("TCP") and paquet["TCP"].flags == "S":
              ip_src = paquet["IP"].src
              if ip_src not in brute_force_attempts:
                  brute_force_attempts[ip_src] = 0
              brute_force_attempts[ip_src] += 1
              if brute_force_attempts[ip_src] > 5:  # Seuil de détection
                  print(f"Tentative de brute force détectée depuis {ip_src}")

      sniff(prn=detecte_brute_force)
      ```

2. **Détection de l'injection SQL** :
    - **Script de détection** :

      python
      ```
      from scapy.all import sniff

      def detecte_sql_injection(paquet):
          if paquet.haslayer("TCP") and paquet.haslayer("Raw"):
              load = paquet["Raw"].load.decode(errors='ignore')
              if "SELECT" in load or "UNION" in load or "DROP" in load:
                  print(f"Injection SQL détectée : {load}")

      sniff(prn=detecte_sql_injection)
      ```

Mise en place d'un système d'alertes

Exercice :

1. **Créer un système de log pour les alertes** :
    - **Script de log** :

      python
      ```
      import logging

      logging.basicConfig(filename='nids_alerts.log', level=logging.INFO,
                          format='%(asctime)s - %(message)s')

      def log_alert(message):
          logging.info(message)
      ```

2. **Intégrer les alertes au système de détection** :
    - **Exemple d'intégration** :

      python
      ```
      def detecte_brute_force(paquet):
          if paquet.haslayer("TCP") and paquet["TCP"].flags == "S":
              ip_src = paquet["IP"].src
      ```

```
        if ip_src not in brute_force_attempts:
            brute_force_attempts[ip_src] = 0
        brute_force_attempts[ip_src] += 1
        if brute_force_attempts[ip_src] > 5:
            message = f"Tentative de brute force détectée depuis {ip_src}"
            log_alert(message)
            print(message)

sniff(prn=detecte_brute_force)
```

Intégration de l'envoi de rapports chiffrés par mail

Exercice :

1. **Configurer l'envoi de mails en utilisant smtplib** :
    - **Script d'envoi de mail** :

      python
```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.application import MIMEApplication

def envoyer_mail():
    sender = "your_email@example.com"
    receiver = "admin@example.com"
    subject = "Rapport NIDS"
    body = "Veuillez trouver ci-joint le rapport du NIDS."

    msg = MIMEMultipart()
    msg['From'] = sender
    msg['To'] = receiver
    msg['Subject'] = subject
    msg.attach(MIMEText(body, 'plain'))

    filename = 'nids_alerts.log'
    with open(filename, 'rb') as attachment:
        part = MIMEApplication(attachment.read(), Name=filename)
        part['Content-Disposition'] = f'attachment; filename="{filename}"'
        msg.attach(part)

    with smtplib.SMTP('smtp.example.com', 587) as server:
        server.starttls()
        server.login(sender, 'password')
        server.sendmail(sender, receiver, msg.as_string())

# Exécuter cette fonction à la fin de chaque journée ou intervalle souhaité
envoyer_mail()
```

Tests et ajustements du NIDS

Exercice :

- **Effectuer des tests de pénétration** pour vérifier l'efficacité du NIDS.
- **Ajuster les seuils** et les règles de détection en fonction des résultats des tests.

Partie 2 : Finalisation de la Soutenance (1h30)

Objectif : Préparer les diapositives, enregistrer la démonstration du NIDS et finaliser la vidéo de soutenance.

Préparation des diapositives

1. **Diapositives pour la présentation du NIDS** :
    - **Introduction** :
        - Présenter le projet de NIDS, les objectifs et la portée.
        - Matériel et logiciels utilisés (Python, Scapy, Docker, Kubernetes).
    - **Détection des attaques** :
        - Expliquer les méthodes de détection implémentées (brute force, injection SQL).
        - Montrer les scripts de détection et expliquer leur fonctionnement.
2. **Diapositives pour les conclusions et perspectives** :
    - **Conclusion** :
        - Résumer les résultats obtenus.

?

    - Discuter des points forts et des limites du NIDS développé.
- **Perspectives** :
    - Proposer des améliorations futures.
    - Discuter de l'intégration avec d'autres outils de sécurité (par exemple, Honeypot).

Enregistrement de la démonstration du NIDS

Exercice :

1. **Démonstration en direct du NIDS** :
    - Montrer le NIDS en action, capturant et analysant les paquets en temps réel.
    - Démontrer la détection d'attaques spécifiques et la génération d'alertes.
2. **Enregistrement des explications** :
    - Expliquer chaque étape du processus de détection.
    - Utiliser des exemples concrets d'attaques détectées pour illustrer les capacités du NIDS.

Finalisation de la soutenance.

Partie 3 : Rendu du projet et préparation à la soutenance (30 min)

Objectif : Vérifier que tous les éléments du projet sont complets et fonctionnels, et rendre la vidéo de soutenance.

Exercice :

1. **Vérification finale** :
    - S'assurer que le NIDS fonctionne correctement et détecte les attaques comme prévu.
    - Vérifier que tous les scripts et configurations sont bien documentés.
2. **Rendu de la vidéo de soutenance** :
    - Transférer la vidéo finale sur la plateforme de rendu désignée.
    - S'assurer que tous les fichiers nécessaires (scripts, documentation) sont inclus dans le rendu.
3. **Préparation à la soutenance** :
    - Réviser les points clés de la présentation.
    - Préparer les réponses aux questions potentielles sur le NIDS et son fonctionnement.

Conclusion

Cette séance permet de finaliser le développement du NIDS, de préparer la vidéo de soutenance et de vérifier que tous les éléments du projet sont complets et fonctionnels. Les étudiants seront prêts à présenter leur projet de manière professionnelle et convaincante.



dépot du diaporama de soutenance et du script d'attaque et du script NIDS

Ouvert le : mercredi 17 décembre 2025, 10:00 **À rendre :** dimanche 21 décembre 2025, 23:59

?